Example:
Test Case 4: Technical Officer Evaluation Test

Code

```
import os
import sys
import pathlib
import time
import pytest
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.chrome.options import Options

# Paths and configuration
BASE_DIR = pathlib.Path(__file__).resolve().parent.parent
if str(BASE_DIR) not in sys.path:
    sys.path.insert(0, str(BASE_DIR))

SCREENSHOT_DIR = BASE_DIR / "test-results" / "screenshots"
SCREENSHOT_DIR.mkdir(parents=True, exist_ok=True)

# Test Configuration - configurable via environment variables
BASE_URL = os.environ.get("BASE_URL", "http://localhost:3000")
LOGIN_URL = f"{BASE_URL}/login"
TO_DASHBOARD_URL = f"{BASE_URL}/technical-officer/dashboard"
TO_EVALUATIONS_URL = f"{BASE_URL}/technical-officer/evaluations"
TO_USERNAME = os.environ.get("TEST_TO_USERNAME", "test_to")
TO_PASSWORD = os.environ.get("TEST_TO_PASSWORD", "TestTO@2026")
HEADLESS_MODE = os.environ.get("HEADLESS", "true").lower() in ("true", "1", "yes")


def setup_test_requisition_for_to():
    """Ensure a valid test requisition in PENDING_TECHNICAL_EVALUATION state is assigned to test_to."""
    import django
    os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'backend.settings')
    django.setup()
    from accounts.models import User, Department, Role
    from procurement.models import Requisition, RequisitionItem

    to_role = Role.objects.get(name='Technical Officer')
    to_user, _ = User.objects.get_or_create(
        username=TO_USERNAME,
        defaults={
            'employee_id': 'TO999',
            'first_name': 'Test',
            'last_name': 'TechnicalOfficer',
            'email': f'{TO_USERNAME}@procuramed.local',
            'role': to_role,
            'technical_specializations': ['Biomedical Equipment', 'Medical, Surgical & Critical Care Equipment'],
            'is_active': True,
            'first_login': False
        }
    )
    to_user.set_password(TO_PASSWORD)
    to_user.technical_specializations = ['Biomedical Equipment', 'Medical, Surgical & Critical Care Equipment']
    to_user.first_login = False
    to_user.is_active = True
    to_user.save()

    # Look for an existing requisition awaiting technical evaluation assigned to test_to
    req = Requisition.objects.filter(
        assigned_technical_officer=to_user,
        status='PENDING_TECHNICAL_EVALUATION'
    ).first()

    if not req:
        # Check any pending technical evaluation requisition and reassign to test_to
        req = Requisition.objects.filter(status='PENDING_TECHNICAL_EVALUATION').first()
        if req:
```

```
            req.assigned_technical_officer = to_user
            req.save()
        else:
            # Create a dedicated clean test requisition in PENDING_TECHNICAL_EVALUATION status
            staff_user = User.objects.filter(role__name='Department Staff').first()
            po_user = User.objects.filter(role__name='Purchase Officer').first()
            dept = Department.objects.filter(name='Medical & Clinical Services').first() or Department.objects.first()
            count = Requisition.objects.count() + 1
            req = Requisition.objects.create(
                req_number=f"REQ-2026-{count:05d}",
                title="Automated Test - Patient Monitor Technical Evaluation",
                priority="HIGH",
                justification="Urgent clinical requirement for Operation Theatre monitoring requiring technical inspection.",
                department=dept,
                requested_by=staff_user,
                assigned_purchase_officer=po_user,
                assigned_technical_officer=to_user,
                status="PENDING_TECHNICAL_EVALUATION",
                requires_technical_evaluation=True,
                technical_specialization="Biomedical Equipment",
                estimated_budget=45000.00
            )
            RequisitionItem.objects.create(
                requisition=req,
                item_name="Multiparameter Patient Monitor",
                category="Biomedical Equipment",
                quantity=2,
                estimated_unit_price=22500.00,
                specifications="High-resolution vitals monitor with ECG, SpO2, and NIBP."
            )
    return req


class TestTechnicalOfficerEvaluation:
    """
    Test Case 4: Technical Officer Evaluation Test Suite
    Validates reviewing assigned purchase requests awaiting technical evaluation,
    assessing item specifications, outcome validation rules, submitting technical evaluation,
    and verifying workflow status transitions.
    """

    @classmethod
    def setup_class(cls):
        """Prepare test requisition in database assigned to test_to."""
        cls.target_req = setup_test_requisition_for_to()

    def setup_method(self, method):
        """Setup Chrome WebDriver before each test execution."""
        options = Options()
        if HEADLESS_MODE:
            options.add_argument("--headless=new")
        options.add_argument("--window-size=1920,1080")
        options.add_argument("--no-sandbox")
        options.add_argument("--disable-dev-shm-usage")
        options.add_argument("--disable-gpu")
        options.add_argument("--ignore-certificate-errors")

        self.driver = webdriver.Chrome(options=options)
        self.driver.maximize_window()
        self.wait = WebDriverWait(self.driver, 12)

    def teardown_method(self, method):
        """Tear down and close the WebDriver session after each test."""
        if hasattr(self, "driver") and self.driver:
            self.driver.quit()

    def click_element(self, element):
        """Helper to reliably scroll and click element across animations."""
        self.driver.execute_script("arguments[0].scrollIntoView({block: 'center'});", element)
        try:
            element.click()
        except Exception:
            self.driver.execute_script("arguments[0].click();", element)
    def login_as_technical_officer(self):
```

```
        """Helper to authenticate as Technical Officer."""
        driver = self.driver
        wait = self.wait

        driver.get(LOGIN_URL)
        username_field = wait.until(EC.visibility_of_element_located((By.NAME, "username")))
        username_field.clear()
        username_field.send_keys(TO_USERNAME)

        password_field = wait.until(EC.visibility_of_element_located((By.NAME, "password")))
        password_field.clear()
        password_field.send_keys(TO_PASSWORD)

        login_btn = wait.until(EC.element_to_be_clickable((By.CSS_SELECTOR, "button[type='submit']")))
        self.click_element(login_btn)

        wait.until(EC.url_contains("/technical-officer/dashboard"))

    def test_technical_officer_evaluation_outcome_validation(self):
        """
        Negative Test: Attempt submitting technical evaluation without selecting
        an Evaluation Outcome (Technically Suitable vs. Return for Clarification).
        Verify validation error alert banner is displayed.
        """
        driver = self.driver
        wait = self.wait

        self.login_as_technical_officer()
        driver.get(TO_EVALUATIONS_URL)
        wait.until(EC.url_contains("/technical-officer/evaluations"))
        wait.until(EC.visibility_of_element_located((By.TAG_NAME, "table")))

        # Locate target requisition row
        row = wait.until(
            EC.visibility_of_element_located((By.XPATH, f"//tr[contains(., '{self.target_req.req_number}')]"))
        )
        eval_btn = row.find_element(By.XPATH, ".//button[contains(., 'Evaluate')]")
        self.click_element(eval_btn)

        # Wait for modal
        wait.until(EC.visibility_of_element_located((By.XPATH, f"//h2[contains(., '{self.target_req.title}')]")))

        # Fill remarks first to satisfy HTML5 textarea required attribute, but omit outcome selection
        remarks_input = wait.until(EC.visibility_of_element_located((By.ID, "tech-remarks")))
        remarks_input.clear()
        remarks_input.send_keys("Specifications verified.")

        # Click submit button
        submit_btn = wait.until(EC.element_to_be_clickable((By.XPATH, "//button[@type='submit']")))
        self.click_element(submit_btn)

        # Verify validation error banner
        err_banner = wait.until(
            EC.visibility_of_element_located((By.XPATH, "//div[contains(@class, 'bg-rose-50')]"))
        )
        driver.save_screenshot(str(SCREENSHOT_DIR / "tc4_validation_error.png"))

        assert "Please select an Evaluation Outcome" in err_banner.text, (
            f"Expected outcome validation error, got: {err_banner.text}"
        )

        # Close modal
        close_btn = driver.find_element(By.XPATH, "//div[contains(@class, 'fixed')]//button[contains(@title, 'Close') or ./*[name()=
        self.click_element(close_btn)

    def test_technical_officer_evaluation_submit_success(self):
        """
        Positive Test: Complete technical evaluation for eligible purchase requisition.
        Verify:
        1. Login as Technical Officer and navigate to Dashboard.
        2. Open Technical Evaluations queue.
        3. Select pending requisition and open Evaluation modal.
        4. Review requisition details, line items, and clinical justification.
        5. Select Evaluation Outcome: Technically Suitable (Approve).
```

```
6. Enter detailed Technical Assessment Remarks.
7. Submit evaluation and verify success confirmation banner.
8. Navigate to Completed tab and verify updated evaluation entry.
9. Verify backend requisition status updated to PENDING_COMMITTEE_REVIEW.
"""
driver = self.driver
wait = self.wait

# Step 1: Login and verify Technical Officer Dashboard
self.login_as_technical_officer()
driver.save_screenshot(str(SCREENSHOT_DIR / "tc4_step1_to_dashboard.png"))

# Step 2: Open Technical Evaluations queue
driver.get(TO_EVALUATIONS_URL)
wait.until(EC.url_contains("/technical-officer/evaluations"))
wait.until(EC.visibility_of_element_located((By.TAG_NAME, "table")))
driver.save_screenshot(str(SCREENSHOT_DIR / "tc4_step2_evaluations_queue.png"))

# Step 3: Locate target requisition row and click Evaluate
row = wait.until(
    EC.visibility_of_element_located((By.XPATH, f"//tr[contains(., '{self.target_req.req_number}')]"))
)
eval_btn = row.find_element(By.XPATH, ".//button[contains(., 'Evaluate')]")
self.click_element(eval_btn)

# Step 4: Verify details in Evaluation modal
modal_title = wait.until(
    EC.visibility_of_element_located((By.XPATH, f"//h2[contains(., '{self.target_req.title}')]"))
)
assert modal_title.is_displayed(), "Requisition title not displayed in modal"
driver.save_screenshot(str(SCREENSHOT_DIR / "tc4_step3_evaluation_modal_details.png"))

# Step 5: Select Evaluation Outcome: Technically Suitable (Approve)
suitable_opt = wait.until(EC.element_to_be_clickable(
    (By.XPATH, "//div[contains(., 'Technically Suitable (Approve)') and contains(@class, 'cursor-pointer')]")
))
self.click_element(suitable_opt)
driver.save_screenshot(str(SCREENSHOT_DIR / "tc4_step4_outcome_selected.png"))

# Step 6: Enter Technical Assessment Remarks
remarks_input = wait.until(EC.visibility_of_element_located((By.ID, "tech-remarks")))
remarks_input.clear()
assessment_text = (
    "Technical specifications verified. The multiparameter patient monitor meets hospital safety standards, "
    "clinical electrical safety parameters (IEC 60601-1), and required monitoring channels. "
    "Forwarded to Procurement Committee for review."
)
remarks_input.send_keys(assessment_text)
driver.save_screenshot(str(SCREENSHOT_DIR / "tc4_step5_assessment_remarks_entered.png"))

# Step 7: Submit Evaluation
submit_btn = wait.until(EC.element_to_be_clickable((By.XPATH, "//button[@type='submit']")))
self.click_element(submit_btn)

# Step 8: Verify Success Notification Banner
succ_banner = wait.until(
    EC.visibility_of_element_located((By.XPATH, "//div[contains(@class, 'bg-emerald-50')]"))
)
driver.save_screenshot(str(SCREENSHOT_DIR / "tc4_step6_success_confirmation.png"))
assert any(term in succ_banner.text.lower() for term in ["technically suitable", "submitted successfully", "recorded"]), (
    f"Expected evaluation success confirmation, got: {succ_banner.text}"
)

# Wait for modal auto-close and switch to Completed tab
time.sleep(2)
driver.get(f"{TO_EVALUATIONS_URL}?tab=completed")
wait.until(EC.url_contains("tab=completed"))
time.sleep(1)
driver.save_screenshot(str(SCREENSHOT_DIR / "tc4_step7_completed_evaluations_tab.png"))

# Step 9: Verify Requisition status updated to PENDING_COMMITTEE_REVIEW in Database
self.target_req.refresh_from_db()
assert self.target_req.status == "PENDING_COMMITTEE_REVIEW", (
    f"Expected status PENDING_COMMITTEE_REVIEW, got: {self.target_req.status}"
```

)

Screenshot

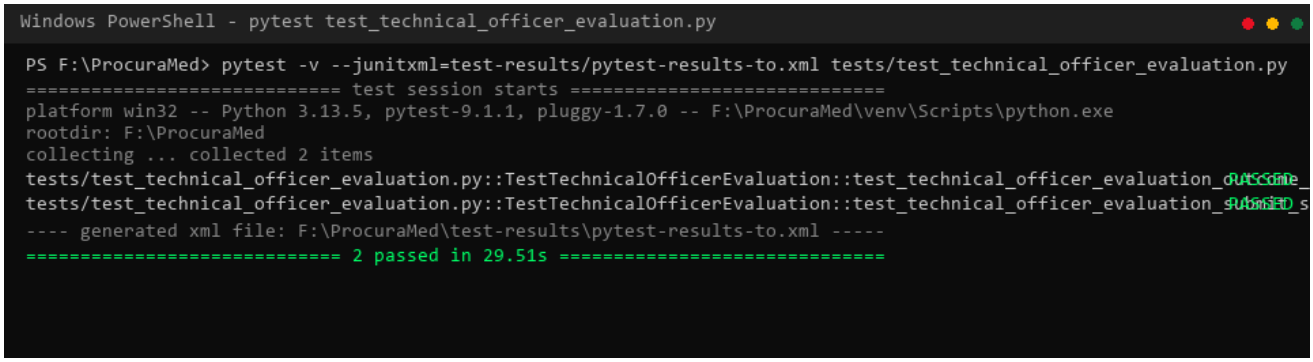


Figure 1 : Technical Officer Evaluation Test successfully (Pytest Execution Results)

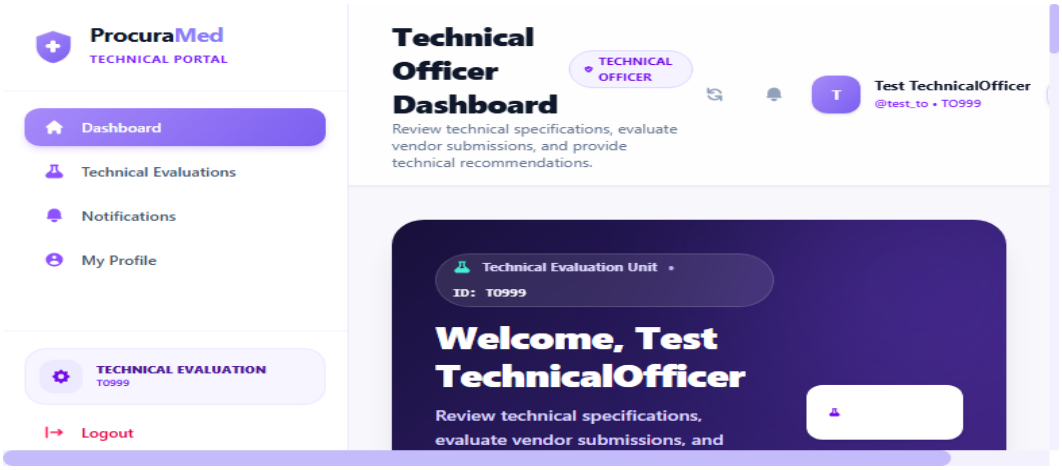


Figure 2 : Technical Officer Dashboard with Key Evaluation Metrics and Assigned Specializations

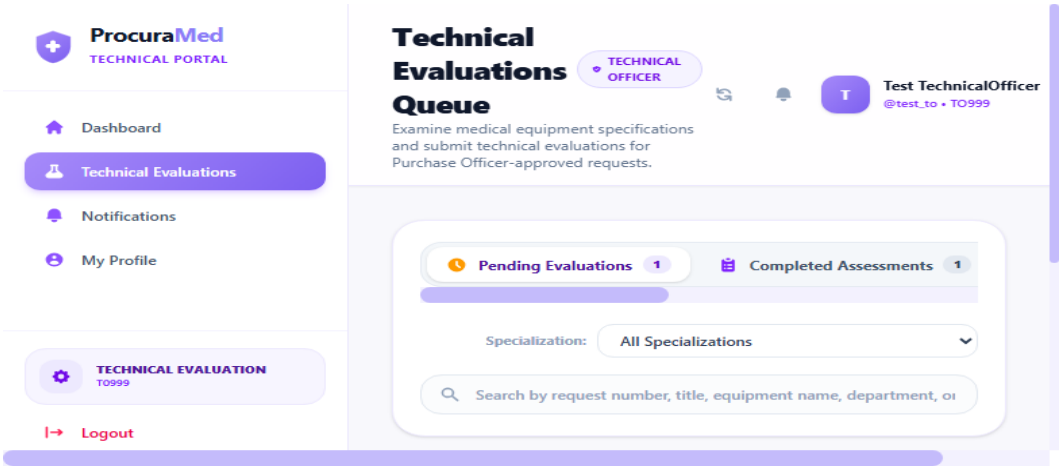


Figure 3 : Technical Evaluations Queue showing Requisitions Pending Assessment



Figure 4 : Technical Evaluation Modal displaying Requisition Details, Line Items, and Clinical Justification

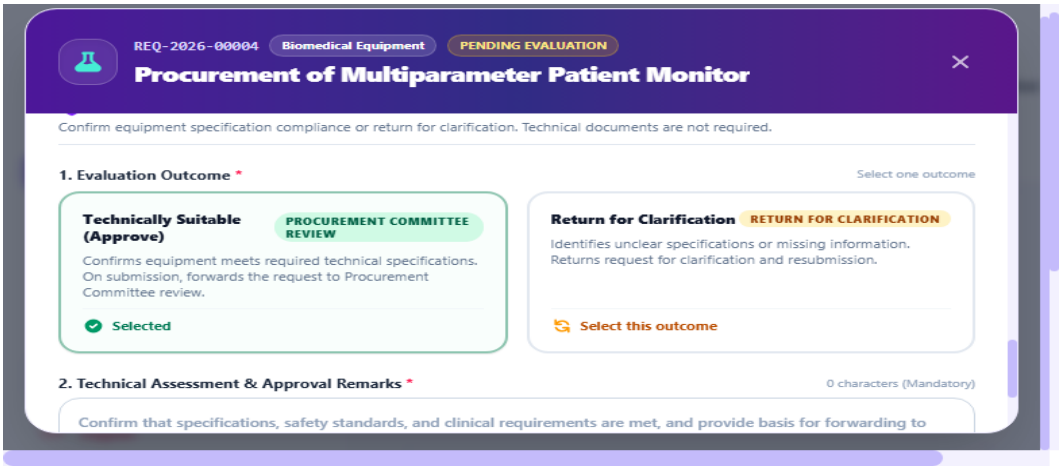


Figure 5 : Evaluation Outcome Selected (Technically Suitable - Approve)

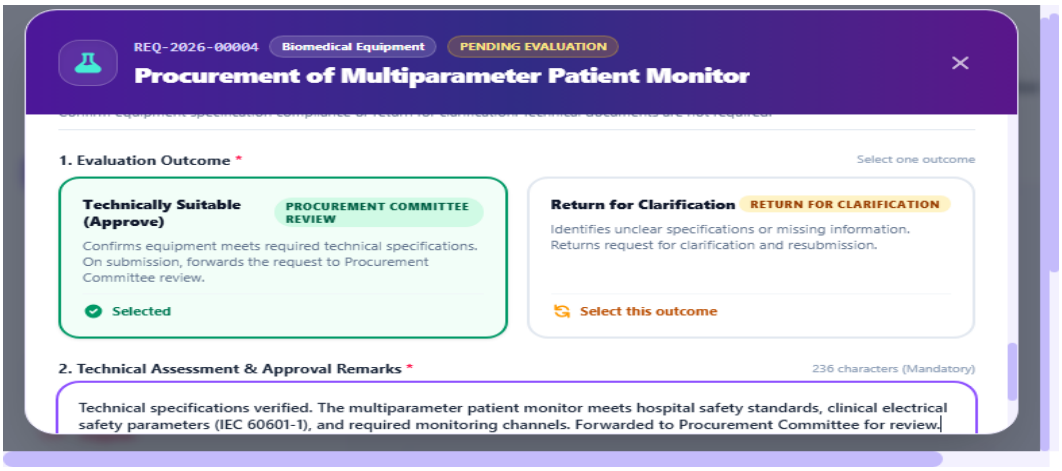


Figure 6 : Technical Assessment Remarks and Specification Compliance Verification Entered

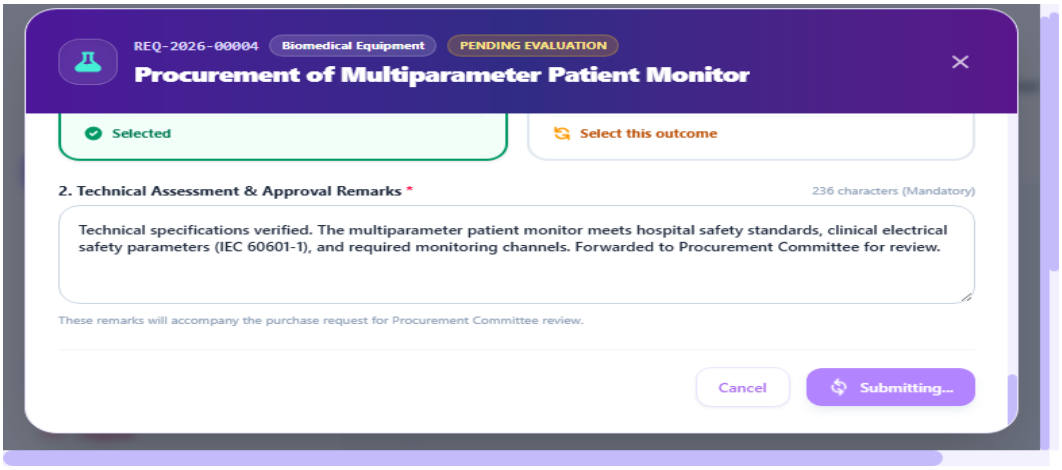


Figure 7 : Technical Evaluation Submitted Successfully Notification Banner

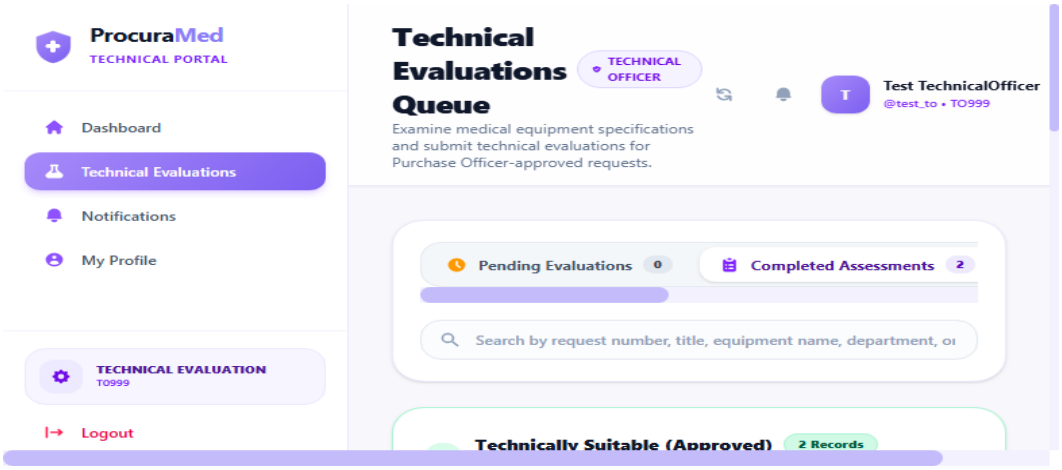


Figure 8 : Verified Technical Evaluation Entry with Status 'Technically Suitable' in Completed History Tab

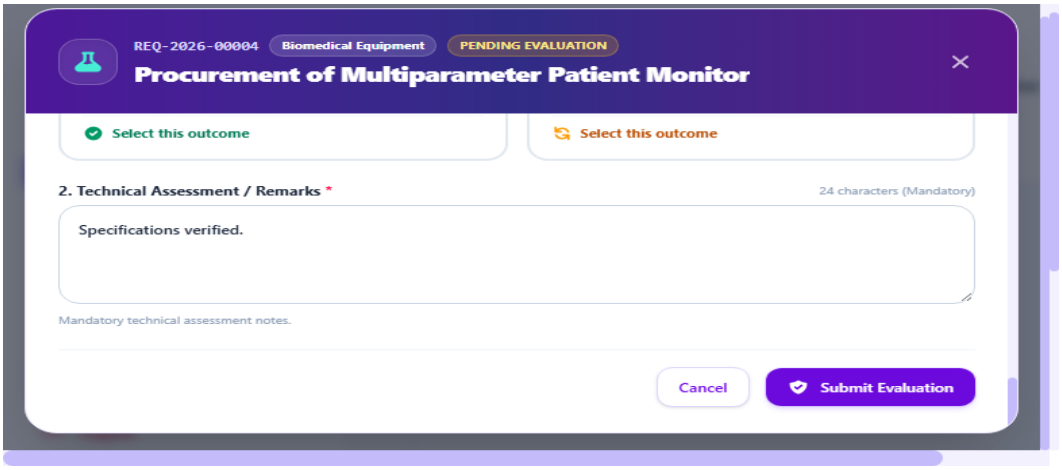


Figure 9 : Mandatory Evaluation Outcome Selection Validation Alert Banner

Test Report
Test Case 4

Project Name: ProcuraMed	
Technical Officer Evaluation Test Case	
Test Case ID: Test_4	**Test Designed By:** QA Automation Engineer
Test Priority(Low/Medium/High): High	**Test Designed Date:** 22/10/2026
Module Name: Technical Evaluation	**Test Executed By :** QA Automation Engineer
Test Title : verify technical officer evaluation of assigned purchase requisition, specification assessment, and approval progression.	**Test Execution Date:** 22/10/2026
Description: Technical Officer logs in, reviews assigned purchase requisition specifications and clinical parameters, assesses technical suitability, enters assessment remarks, and approves request for Procurement Committee review.	
Pre-Condition : Authorized Technical Officer is logged in and has eligible purchase requisitions in status 'Pending Technical Evaluation' in their queue.	

Step	Test Step	Test Data	Expected Result	Actual Result	Status(Pass/Fail)
1	Log in as Technical Officer and navigate to Technical Evaluations queue	URL: /technical-officer/evaluations User: test_to	Display pending technical evaluations assigned to the Technical Officer	Technical evaluations queue loaded showing assigned purchase requests with tracking numbers and department	**pass**
2	Select eligible requisition and open Technical Evaluation modal	Requisition: REQ-2026-00004 Title: Procurement of Multiparameter Patient Monitor	Open Technical Evaluation modal displaying requisition details, line items, and evaluation form	Evaluation modal opened showing read-only metadata, requested items, budget (■45,000.00), and clinical justification	**pass**
3	Review item specifications and clinical standards compliance	Item: Multiparameter Patient Monitor (Qty: 2) Category: Biomedical Equipment	Specifications and clinical parameters verified against hospital technical standards	Item specifications reviewed; confirmed vitals monitoring parameters and IEC 60601-1 compliance	**pass**
4	Select Evaluation Outcome: Technically Suitable (Approve)	Outcome: Technically Suitable (Approve)	Outcome selected; action button updates to 'Approve & Forward to Committee'	Outcome selected; UI reflects selection with emerald border and approval action tag	**pass**
5	Enter detailed Technical Assessment Remarks and click Submit	Remarks: 'Technical specifications verified. The multiparameter patient monitor meets hospital safety standards... Forwarded to Procurement Committee.'	Evaluation submitted; confirmation banner displayed: 'Technical evaluation submitted successfully... forwarded to Procurement Committee review'	Confirmation banner displayed; evaluation recorded successfully; modal auto-closes	**pass**
6	Navigate to Completed tab and verify updated evaluation record	Tab: Completed Requisition: REQ-2026-00004	Requisition record displayed under Completed tab with recommendation 'Technically Suitable'	Requisition displayed in Completed History with recommendation badge 'Technically Suitable' and status 'Pending Committee Review'	**pass**

Post-Condition: TechnicalEvaluation record saved, requisition status updated to 'PENDING_COMMITTEE_REVIEW' in database, approval entry created, and requisition routed to Procurement Committee queue.

Test Report – Negative Test Cases
Test Case 4.1: Outcome Selection Validation

Project Name: ProcuraMed					
Technical Officer Evaluation Validation Test Cases					
Test Case ID: Test_4_Neg			**Test Designed By:** QA Automation Engineer		
Test Priority(Low/Medium/High): Medium			**Test Designed Date:** 22/10/2026		
Module Name: Technical Evaluation			**Test Executed By :** QA Automation Engineer		
Test Title : verify validation error when submitting evaluation without selecting mandatory outcome.			**Test Execution Date:** 22/10/2026		
Description: Verify system enforces mandatory selection of an Evaluation Outcome (Technically Suitable or Return for Clarification) before permitting evaluation submission.					
Pre-Condition : Technical Officer has opened evaluation modal for an assigned requisition.					
Step	**Test Step**	**Test Data**	**Expected Result**	**Actual Result**	**Status(Pass/Fail)**
1	Open Technical Evaluation modal for assigned requisition	Requisition: REQ-2026-00004	Display Technical Evaluation modal with unselected outcome choices	Modal displayed showing evaluation outcome options unselected	**pass**
2	Enter assessment remarks and click Submit without selecting Outcome	Remarks: 'Specifications verified.' Outcome: [Unselected]	Validation error banner displayed: 'Please select an Evaluation Outcome: "Technically Suitable (Approve)" or "Return for Clarification".'	Validation error banner displayed as expected; submission blocked; status unchanged	**pass**
Post-Condition: Evaluation submission is blocked; requisition status remains 'PENDING_TECHNICAL_EVALUATION'; data integrity preserved.					